

1594. Maximum Non Negative Product in a Matrix Solved

Medium

Topics

Companies

Hint

You are given a $m \times n$ matrix `grid`. Initially, you are located at the top-left corner $(0, 0)$, and in each step, you can only **move right or down** in the matrix.

Among all possible paths starting from the top-left corner $(0, 0)$ and ending in the bottom-right corner $(m - 1, n - 1)$, find the path with the **maximum non-negative product**. The product of a path is the product of all integers in the grid cells visited along the path.

Return the *maximum non-negative product modulo* $10^9 + 7$. If the maximum product is **negative**, return -1 .

Notice that the modulo is performed after getting the maximum product.

Example 1:

-1	-2	-3
-2	-3	-3
-3	-3	-2

Input: `grid = [[-1,-2,-3],[-2,-3,-3],[-3,-3,-2]]`

Output: `-1`

Explanation: It is not possible to get non-negative product in the path from $(0, 0)$ to $(2, 2)$, so return -1 .

Example 2:

1	-2	1
1	-2	1
3	-4	1

Input: `grid = [[1,-2,1],[1,-2,1],[3,-4,1]]`

Output: 8

Explanation: Maximum non-negative product is shown ($1 * 1 * -2 * -4 * 1 = 8$).

Example 3:

1	3
0	-4

Input: `grid = [[1,3],[0,-4]]`

Output: 0

Explanation: Maximum non-negative product is shown ($1 * 0 * -4 = 0$).

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 15`
- `-4 <= grid[i][j] <= 4`

Python:

class Solution:

def maxProductPath(self, grid):

 m, n = len(grid), len(grid[0])

 MOD = 10**9 + 7

 maxProd = [[0]*n for _ in range(m)]

 minProd = [[0]*n for _ in range(m)]

 maxProd[0][0] = minProd[0][0] = grid[0][0]

 for i in range(1, m):

 maxProd[i][0] = minProd[i][0] = maxProd[i-1][0] * grid[i][0]

 for j in range(1, n):

 maxProd[0][j] = minProd[0][j] = maxProd[0][j-1] * grid[0][j]

```

for i in range(1, m):
    for j in range(1, n):
        val = grid[i][j]

        candidates = [
            maxProd[i-1][j] * val,
            minProd[i-1][j] * val,
            maxProd[i][j-1] * val,
            minProd[i][j-1] * val
        ]

        maxProd[i][j] = max(candidates)
        minProd[i][j] = min(candidates)

ans = maxProd[m-1][n-1]
return -1 if ans < 0 else ans % MOD

```

JavaScript:

```

var maxProductPath = function(grid) {
    const m = grid.length, n = grid[0].length;
    const MOD = 1e9 + 7;

    let maxProd = Array.from({length: m}, () => Array(n).fill(0));
    let minProd = Array.from({length: m}, () => Array(n).fill(0));

    maxProd[0][0] = minProd[0][0] = grid[0][0];

    for (let i = 1; i < m; i++) {
        maxProd[i][0] = minProd[i][0] = maxProd[i-1][0] * grid[i][0];
    }

    for (let j = 1; j < n; j++) {
        maxProd[0][j] = minProd[0][j] = maxProd[0][j-1] * grid[0][j];
    }

    for (let i = 1; i < m; i++) {
        for (let j = 1; j < n; j++) {
            let val = grid[i][j];

            let candidates = [
                maxProd[i-1][j] * val,
                minProd[i-1][j] * val,
                maxProd[i][j-1] * val,
                minProd[i][j-1] * val
            ]

```

```

];

    maxProd[i][j] = Math.max(...candidates);
    minProd[i][j] = Math.min(...candidates);
}
}

let ans = maxProd[m-1][n-1];
return ans < 0 ? -1 : ans % MOD;
};

```

Java:

```

class Solution {
    public int maxProductPath(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        long MOD = 1000000007;

        long[][] maxProd = new long[m][n];
        long[][] minProd = new long[m][n];

        maxProd[0][0] = minProd[0][0] = grid[0][0];

        for (int i = 1; i < m; i++) {
            maxProd[i][0] = minProd[i][0] = maxProd[i - 1][0] * grid[i][0];
        }

        for (int j = 1; j < n; j++) {
            maxProd[0][j] = minProd[0][j] = maxProd[0][j - 1] * grid[0][j];
        }

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                long val = grid[i][j];

                long a = maxProd[i-1][j] * val;
                long b = minProd[i-1][j] * val;
                long c = maxProd[i][j-1] * val;
                long d = minProd[i][j-1] * val;

                maxProd[i][j] = Math.max(Math.max(a,b), Math.max(c,d));
                minProd[i][j] = Math.min(Math.min(a,b), Math.min(c,d));
            }
        }
    }
}

```

```
    long ans = maxProd[m-1][n-1];  
    if(ans < 0) return -1;  
    return (int)(ans % MOD);  
}  
}
```